Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

ANALYSIS AND EVALUATION OF CONGESTION-AVOIDANCE MECHANISMS (TAHOE, RENO, CUBIC) WITH A PROPOSED LIGHTWEIGHT PREDICTIVE CONGESTION ESTIMATOR

Project Demonstration – Slot F1, Team 1

24BCE0780	**VARSHINI HARIKRISHNA**
24BCE2219	**ANUSKA SEN**
24BDS0066	**ANTRA**

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

GitHub: https://github.com/Antra1705/F1_Team1

DEMONSTRATION OVERVIEW

Aim: To demonstrate, with live simulations, that **Predictive TCP** – a TCP sender that runs a four-leaf decision tree every 100 ms to predict bottleneck congestion **0.5 s ahead** – loses far fewer packets and adds far less queueing delay than standard TCP variants, while keeping almost the same throughput, and to show its main limitation openly. **Tools:** NS-3.48 network simulator (C++), Python analysis pipeline and a Streamlit interactive simulator. **Every run in this document uses seed 4, a seed that was never used for training or tuning.**

References: Project report – Analysis and Evaluation of Congestion-Avoidance Mechanisms (Tahoe, Reno, CUBIC) with a Proposed Lightweight Predictive Congestion Estimator; source code and results – `github.com/Antra1705/F1_Team1`*; Jacobson, SIGCOMM 1988; Brakmo and Peterson, IEEE JSAC 1995 (Vegas); Ha, Rhee and Xu, 2008 (CUBIC); Cardwell et al., ACM Queue 2016 (BBR); ns-3 documentation, nsnam.org.*

SECTION 1: Demonstration Setup

Problem Understanding

Classic TCP variants (Tahoe, Reno, CUBIC) slow down only **after** the router in front of the bottleneck link has dropped a packet. By then the queue is already full: every packet waits in it, which adds delay, and the dropped packets must be sent again. The round-trip time (RTT), however, starts rising as soon as the queue starts filling – well before any loss. Predictive TCP uses this early signal: a decision tree learned from simulation data predicts whether the queue will be full half a second later, and the sender cuts its congestion window by 20% before the overflow happens. This demonstration shows the effect directly, first from the simulator's command line and then in the interactive simulator.

Simulated Network

Figure 1: Simulated Dumbbell Topology (NS-3.48)

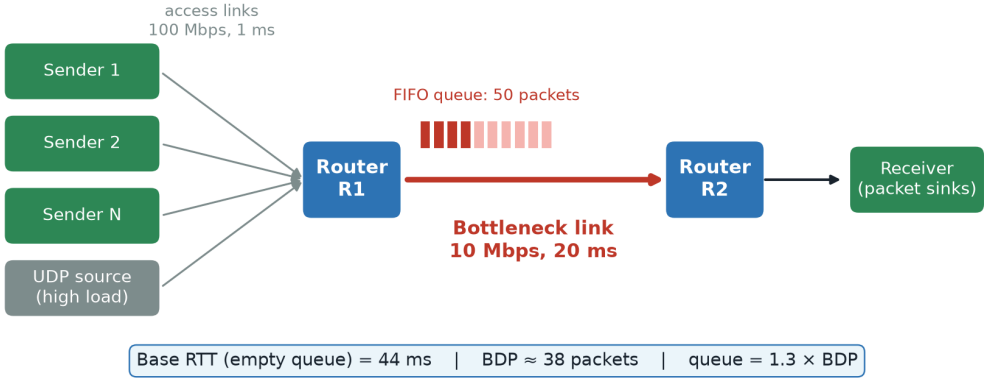


Table 1: Simulation parameters

Parameter	**Value**	**Meaning**
Bottleneck link	10 Mbps, 20 ms one-way delay	The slowest link, where the queue builds up
Access links	100 Mbps, 1 ms	Links from the senders and to the receiver
Router queue	50 packets, FIFO (drop-tail)	A full queue drops every new packet
Base RTT	44 ms	Round trip with an empty queue
Run length	60 s of simulated time	About 1–2 s of real computing time
Seeds	10 (1–3 for training and tuning, 4–10 unseen)	Repeatable traffic patterns; seed 4 is used here
Variants compared	Tahoe, Reno, CUBIC, Vegas, BBR	Against the proposed Predictive TCP

Table 2: Congestion levels

Level	**Traffic**	**Load**
Low	3 on/off TCP flows at 2 Mbps	About 30%
Medium	6 on/off TCP flows at 3 Mbps	About 90%, bursty
High (used in this demo)	8 long-lived TCP flows + 2 Mbps on/off UDP	Saturated

Launching the Demonstration

The simulator is compiled once with `./build.sh`. The demonstration then uses two terminals: one runs the interactive simulator, the other runs single NS-3 simulations.

```
# Terminal 1: start the interactive simulator (opens http://localhost:8501)
cd ~/Desktop/F1_Team1 && ./app/run_app.sh

# Terminal 2: prepare an output folder for single simulations
mkdir -p /tmp/demo && cd ~/ns-3
```

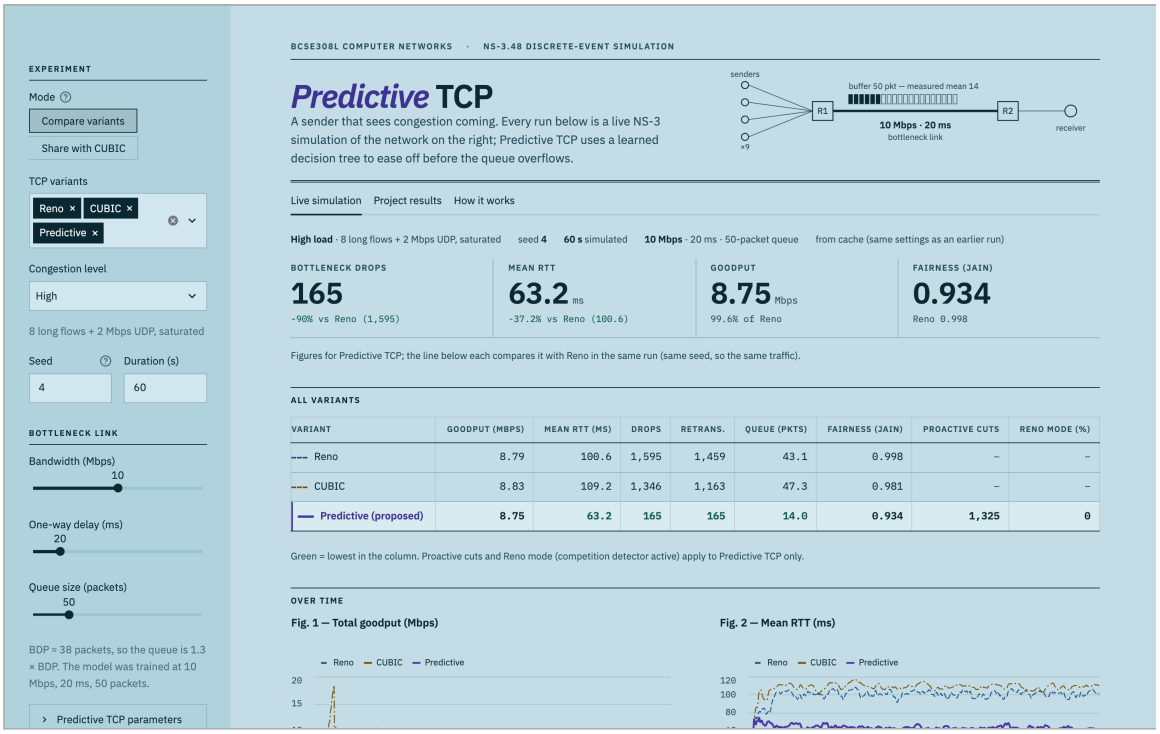


Figure 2: The interactive simulator. Left: experiment settings (variants, load, seed, link and Predictive TCP parameters). Top right: the simulated network, whose queue boxes fill to the measured mean queue. Below: headline numbers, the comparison table and time-series charts.

SECTION 2: Part A – The Simulation in the Terminal

Step 1: Running Reno and Predictive TCP

Procedure: run the same high-load scenario on seed 4 twice, changing only the TCP variant.

```
./ns3 run --no-build \
    "tcp-predictor --variant=reno --level=high --seed=4 --outDir=/tmp/demo"
./ns3 run --no-build \
    "tcp-predictor --variant=predictive --level=high --seed=4 --outDir=/tmp/demo"
```

Output:

```
reno_high_s4: goodput 8.96 Mbps, retransmissions 1459, bottleneck drops 1608,
              proactive reductions 0, competitive mode 0.00%
predictive_high_s4: goodput 8.91 Mbps, retransmissions 165, bottleneck drops 165,
              proactive reductions 1325, competitive mode 0.00%
```

Observation

- Each command simulates 60 s of network time and finishes in about one second.
- Same network, same traffic and same seed – only the congestion-control algorithm differs.
- Reno: **1,608** packets dropped at the bottleneck. Predictive TCP: **165** – about **90% fewer**.
- Goodput is almost unchanged (8.91 vs 8.96 Mbps).

- The **1,325 proactive reductions** are the early cuts: each time the tree predicted a full queue within 0.5 s, the sender reduced its window by 20%.

Step 2: Inside the Simulation – the 100 ms Log

Procedure: every 100 ms the simulator logs, for each flow, its smoothed RTT, minimum RTT and congestion window, together with the real router queue. The command below prints the snapshot of all eight TCP flows at exactly t = 30 s for both runs.

```
for v in reno predictive; do echo "== $v"; awk -F, 'NR==1 || ($1>=30 && $1<30.05)' \
  /tmp/demo/samples_${v}_high_s4.csv | cut -d, -f1-5,10,12 | column -s, -t; done
```

Output:

```
== reno
time    flow  srtt_ms  min_rtt_ms  cwnd_bytes  queue_pkts  drops
30.000  0     96.076   53.293      14842       44          0
30.000  1     101.046  70.823      1448        44          0
30.000  2     93.830   56.556      18824       44          0
30.000  3     90.552   44.000      15566       44          0
30.000  4     90.014   56.707      18100       44          0
30.000  5     103.427  52.000      10136       44          0
30.000  6     91.569   58.500      11588       44          0
30.000  7     90.379   59.540      14480       44          0
== predictive
time    flow  srtt_ms  min_rtt_ms  cwnd_bytes  queue_pkts  drops
30.000  0     65.757   53.412      10193       13          0
30.000  1     66.988   52.708      12268       13          0
30.000  2     60.814   44.000      5792        13          0
30.000  3     62.485   44.000      5792        13          0
30.000  4     61.222   46.000      5792        13          0
30.000  5     63.186   51.483      8398        13          0
30.000  6     65.843   53.174      6950        13          0
30.000  7     61.094   44.000      5792        13          0
```

Observation

- With Reno the router holds **44 of 50 packets** – almost full – so every packet waits and the RTT of the eight flows is **90–103 ms**.
- With Predictive TCP the queue holds only **13 packets**, and the RTT is **61–67 ms**, close to the 44 ms base RTT.
- This log is also the training data of the model. The model learned only from the sender-side RTT columns; the `queue_pkts` column is used only to label what happened 0.5 s later and is never seen by the sender.

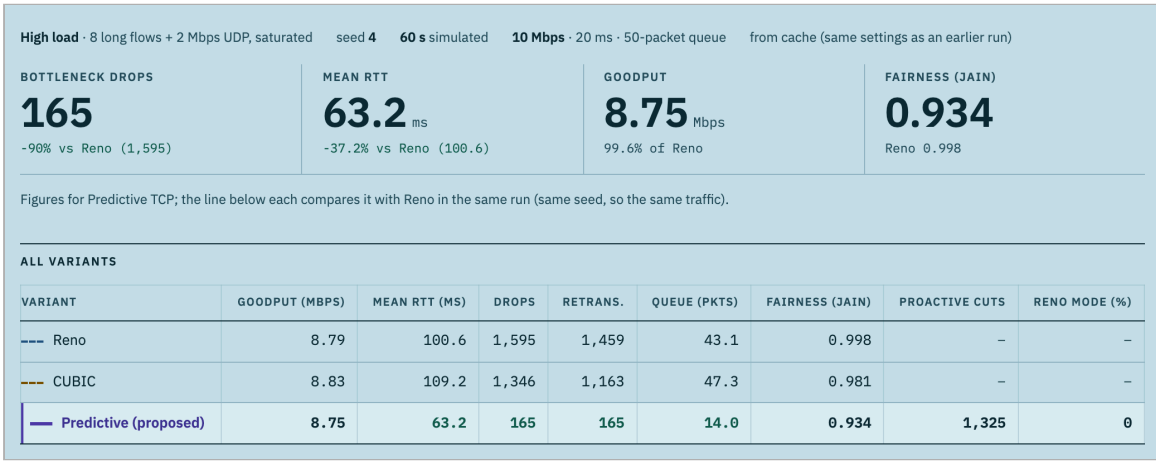
Inference (Part A): with identical traffic, the only change – predicting congestion and backing off 20% early – keeps the bottleneck queue short. A short queue means low delay, and a queue that never overflows means almost no drops, at practically the same goodput.

SECTION 3: Part B – Interactive Demonstration

The interactive simulator runs the same compiled NS-3 program for every button press, so each result below is a real simulation. Settings for Steps 3–6: high load, seed 4, 10 Mbps, 20 ms, unless stated.

Step 3: Main Result at High Load

Procedure: open the app; the default experiment (Reno, CUBIC and Predictive TCP) runs automatically.



High load · 8 long flows + 2 Mbps UDP, saturated seed **4** **60 s** simulated **10 Mbps** · 20 ms · 50-packet queue from cache (same settings as an earlier run)

BOTTLENECK DROPS	MEAN RTT	GOODPUT	FAIRNESS (JAIN)
165	63.2 ms	8.75 Mbps	0.934
-90% vs Reno (1,595)	-37.2% vs Reno (100.6)	99.6% of Reno	Reno 0.998

Figures for Predictive TCP; the line below each compares it with Reno in the same run (same seed, so the same traffic).

ALL VARIANTS

VARIANT	GOODPUT (MBPS)	MEAN RTT (MS)	DROPS	RETRANS.	QUEUE (PKTS)	FAIRNESS (JAIN)	PROACTIVE CUTS	RENO MODE (%)
--- Reno	8.79	100.6	1,595	1,459	43.1	0.998	–	–
--- CUBIC	8.83	109.2	1,346	1,163	47.3	0.981	–	–
— Predictive (proposed)	**8.75**	**63.2**	**165**	**165**	**14.0**	**0.934**	**1,325**	**0**

Figure 3: Headline numbers for Predictive TCP (each compared with Reno in the same run) and the results of all three variants.

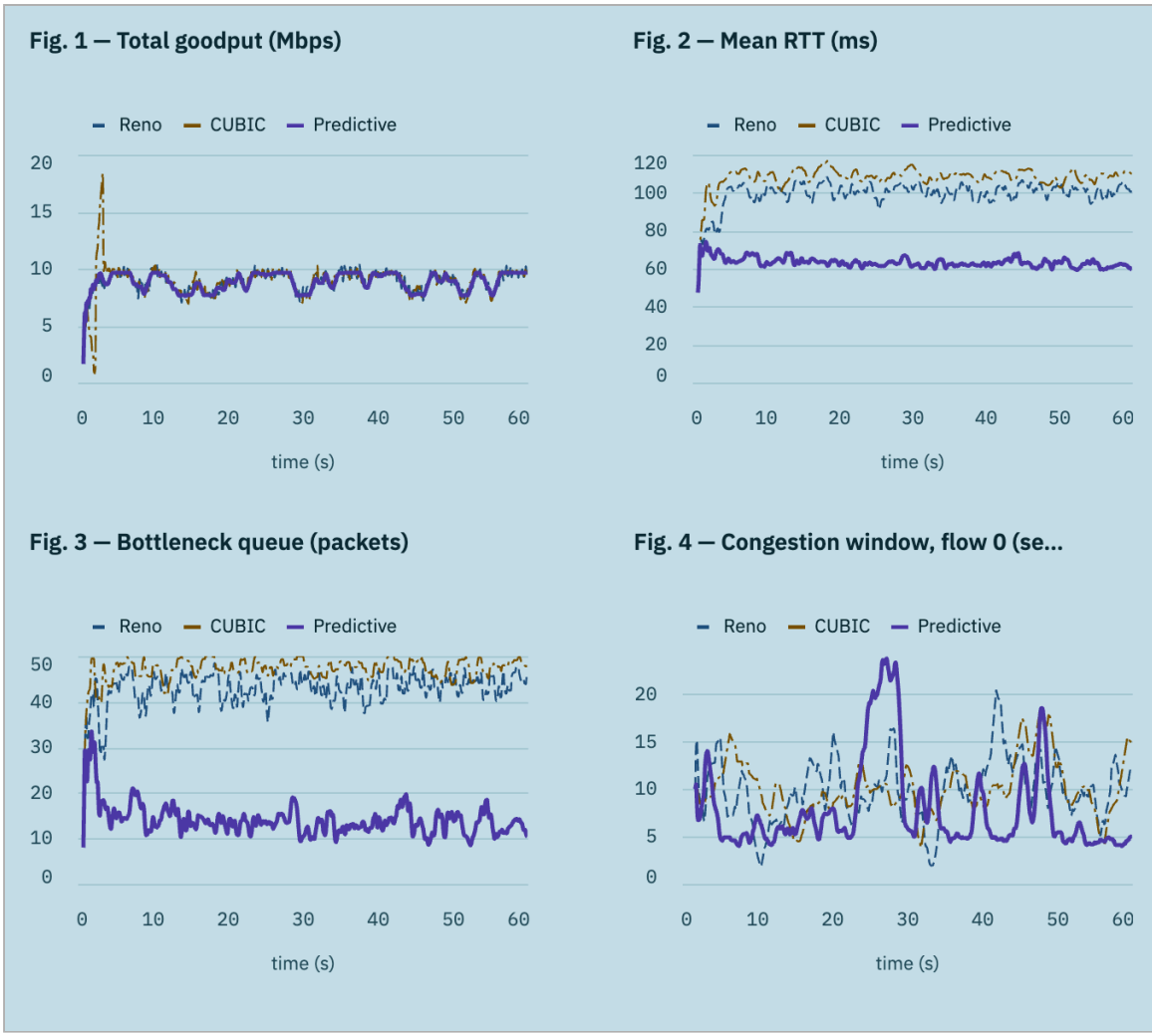


Figure 4: Behaviour over 60 s: goodput (Fig. 1), mean RTT (Fig. 2), bottleneck queue (Fig. 3) and the congestion window of one flow (Fig. 4).

Table 3: Step 3 results (high load, seed 4)

Metric	Reno	CUBIC	Predictive TCP	Change vs Reno
Bottleneck drops	1,595	1,346	**165**	**−90%**
Mean RTT (ms)	100.6	109.2	**63.2**	**−37.2%**
Goodput (Mbps)	8.79	8.83	8.75	99.6% of Reno
Mean queue (packets)	43.1	47.3	**14.0**	−68%
Fairness (Jain)	0.998	0.981	0.934	above the 0.9 target

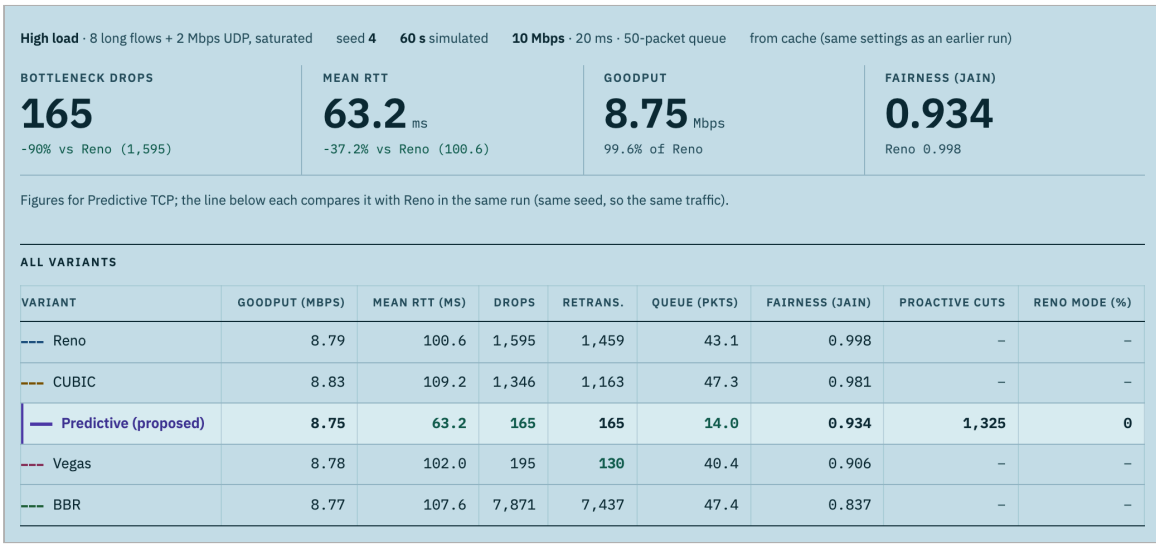
Observation

- **Fig. 3 (queue)** explains the result: Reno and CUBIC keep the queue at 43–47 of 50 packets, while Predictive TCP keeps it around 14.
- **Fig. 2 (RTT):** Predictive TCP stays near 63 ms; Reno and CUBIC stay around 100–110 ms.

- **Fig. 1 (goodput):** the three lines overlap – the early backoff does **not** make the sender slower.
- **Fig. 4 (window):** Reno shows the classic saw-tooth (grow until loss, then halve); Predictive TCP makes small, early cuts instead.

Step 4: Comparison with All Five Baselines

Procedure: in *TCP variants*, add Vegas and BBR, then press *Run simulation.*



High load · 8 long flows + 2 Mbps UDP, saturated seed **4** **60 s** simulated **10 Mbps** · 20 ms · 50-packet queue from cache (same settings as an earlier run)

BOTTLENECK DROPS	MEAN RTT	GOODPUT	FAIRNESS (JAIN)
165	**63.2** ms	**8.75** Mbps	**0.934**
-90% vs Reno (1,595)	-37.2% vs Reno (100.6)	99.6% of Reno	Reno 0.998

Figures for Predictive TCP; the line below each compares it with Reno in the same run (same seed, so the same traffic).

ALL VARIANTS

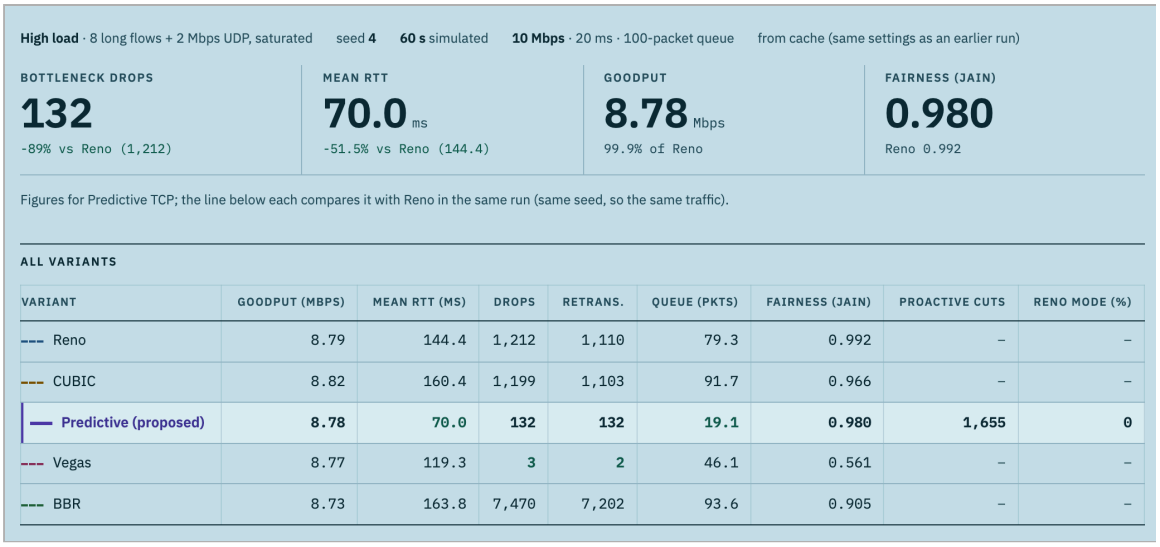
VARIANT	GOODPUT (MBPS)	MEAN RTT (MS)	DROPS	RETRANS.	QUEUE (PKTS)	FAIRNESS (JAIN)	PROACTIVE CUTS	RENO MODE (%)
Reno	8.79	100.6	1,595	1,459	43.1	0.998	–	–
CUBIC	8.83	109.2	1,346	1,163	47.3	0.981	–	–
Predictive (proposed)	**8.75**	**63.2**	**165**	**165**	**14.0**	**0.934**	**1,325**	**0**
Vegas	8.78	102.0	195	**130**	40.4	0.906	–	–
BBR	8.77	107.6	7,871	7,437	47.4	0.837	–	–

Figure 5: All six TCP variants on the same traffic (green = lowest value in each column).

- **BBR** (Google, version 1) floods this small buffer: **7,871 drops**, about five times Reno.
- **Vegas**, also delay-based, has few drops (195) but its RTT is still **102.0 ms** against 63.2 ms for Predictive TCP, and its fairness is lower (0.906).
- Predictive TCP has the **lowest RTT and the lowest queue of all six** variants, and the fewest drops.

Step 5: A Larger Buffer (Bufferbloat)

Procedure: move *Queue size* from 50 to **100 packets** and run again.



High load · 8 long flows + 2 Mbps UDP, saturated seed **4** **60 s** simulated **10 Mbps** · 20 ms · 100-packet queue from cache (same settings as an earlier run)

BOTTLENECK DROPS	MEAN RTT	GOODPUT	FAIRNESS (JAIN)
132	**70.0** ms	**8.78** Mbps	**0.980**
-89% vs Reno (1,212)	-51.5% vs Reno (144.4)	99.9% of Reno	Reno 0.992

Figures for Predictive TCP; the line below each compares it with Reno in the same run (same seed, so the same traffic).

ALL VARIANTS

VARIANT	GOODPUT (MBPS)	MEAN RTT (MS)	DROPS	RETRANS.	QUEUE (PKTS)	FAIRNESS (JAIN)	PROACTIVE CUTS	RENO MODE (%)
Reno	8.79	144.4	1,212	1,110	79.3	0.992	–	–
CUBIC	8.82	160.4	1,199	1,103	91.7	0.966	–	–
Predictive (proposed)	**8.78**	**70.0**	**132**	**132**	**19.1**	**0.980**	**1,655**	**0**
Vegas	8.77	119.3	3	2	46.1	0.561	–	–
BBR	8.73	163.8	7,470	7,202	93.6	0.905	–	–

Figure 6: Results with a 100-packet router queue.

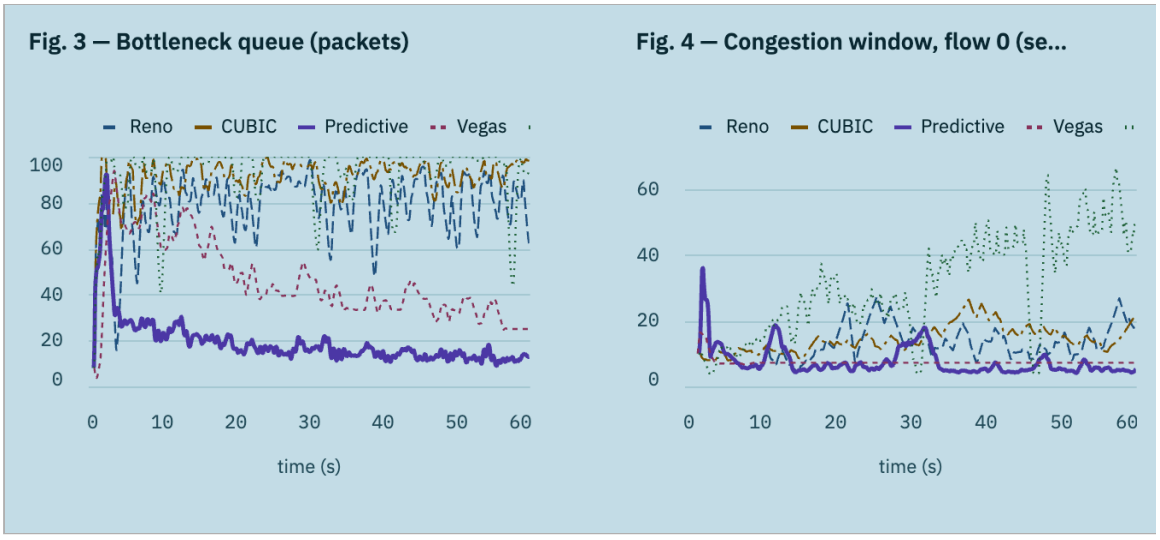


Figure 7: Bottleneck queue and congestion window with a 100-packet queue.

Table 4: Step 5 results (100-packet queue)

Metric	**Reno**	**CUBIC**	**Predictive TCP**	**Change vs Reno**
Mean RTT (ms)	144.4	160.4	**70.0**	**−51.5%**
Bottleneck drops	1,212	1,199	**132**	**−89%**
Mean queue (packets)	79.3	91.7	**19.1**	−76%
Goodput (Mbps)	8.79	8.82	8.78	99.9% of Reno

Loss-based TCP fills whatever buffer exists, so a bigger buffer only adds delay (**bufferbloat**): Reno's RTT rises from 100.6 to 144.4 ms. Predictive TCP keeps the queue short whatever its size, so its RTT barely changes (70.0 ms) and its advantage grows to **51.5%**. The model was trained only with the 50-packet queue, so this also shows that it generalises. Vegas drops only 3 packets here, but its fairness falls to 0.561.

Step 6: Sharing the Link with CUBIC (Main Limitation)

Procedure: set *Mode* to *Share with CUBIC* (4 Predictive TCP flows and 4 CUBIC flows; a fair share is 50%) and run. Then open *Predictive TCP parameters*, switch the *Competition detector* off and run again.

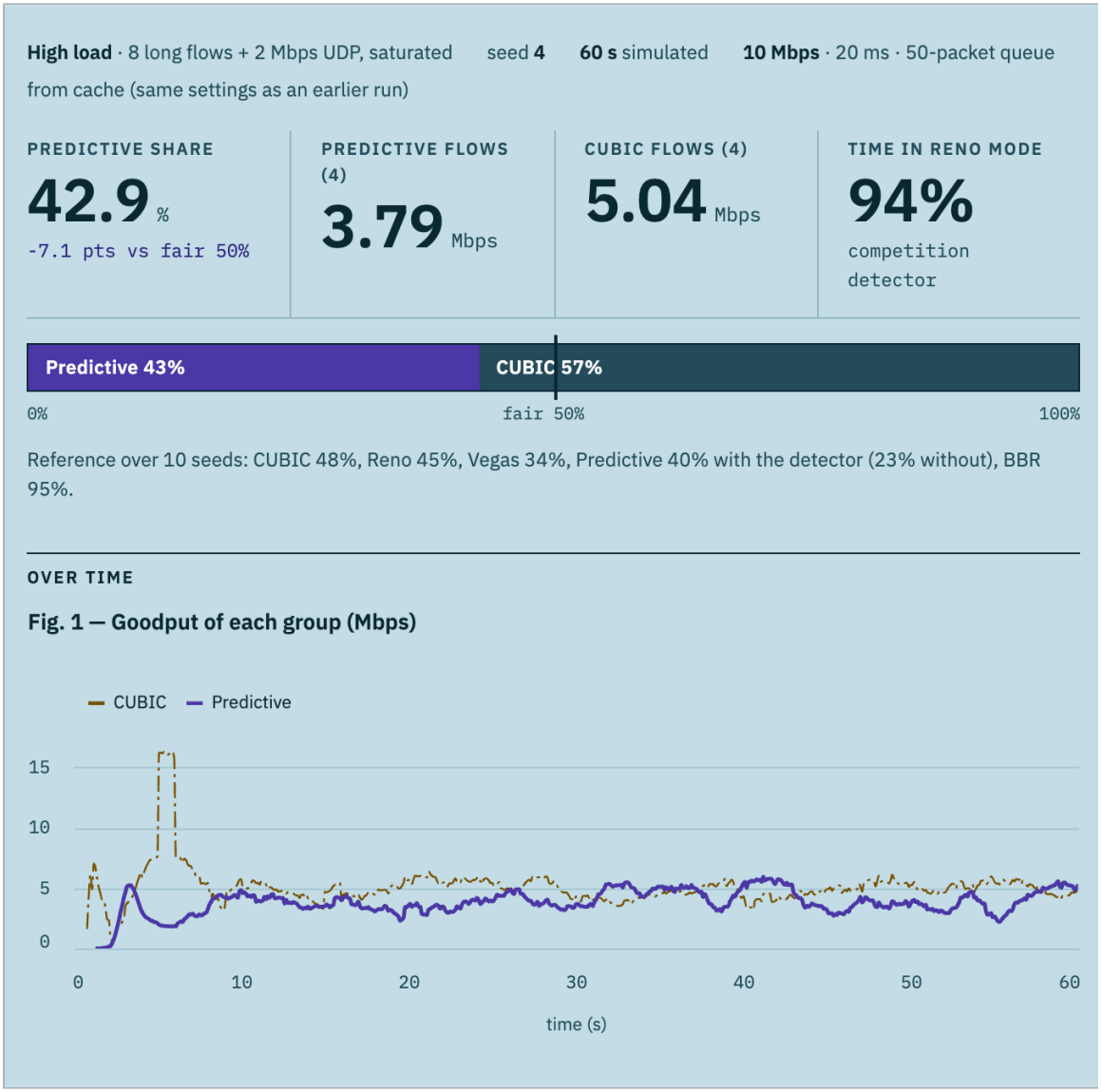


Figure 8: Sharing with CUBIC, competition detector on.

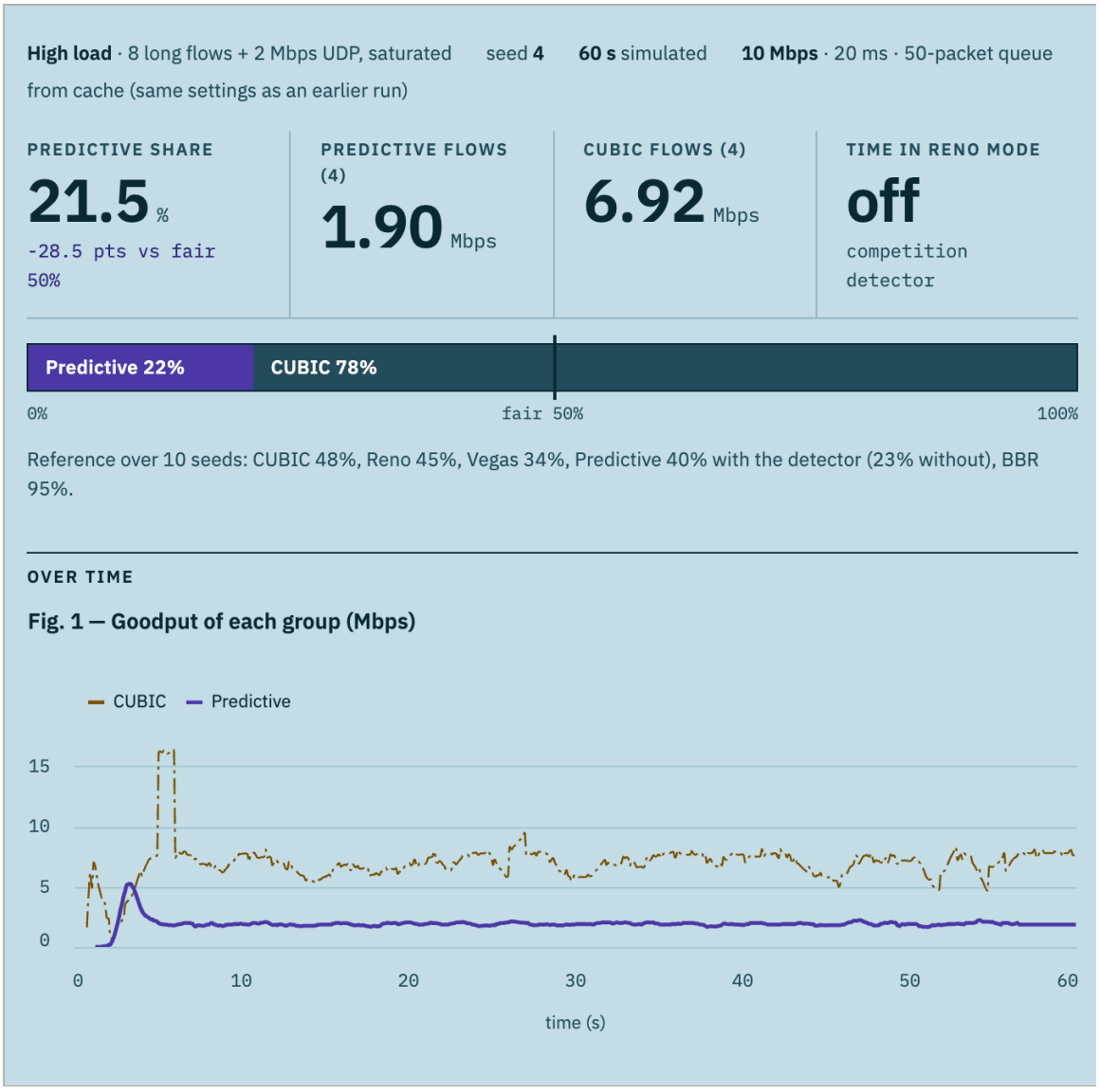


Figure 9: Sharing with CUBIC, competition detector off.

Table 5: Step 6 results (fair share = 50%)

	Detector on	**Detector off**
Predictive TCP share of goodput	**42.9%**	21.5%
Predictive TCP flows (4)	3.79 Mbps	1.90 Mbps
CUBIC flows (4)	5.04 Mbps	6.92 Mbps
Time in Reno mode	94%	off

CUBIC slows down only after a loss, so it keeps the queue full. Without help, Predictive TCP reads the full queue as congestion coming and keeps backing off, ending with about 22% of the link. The **competition detector** notices that the lowest RTT over 2 s stays above 1.4 × the base RTT – the queue never drains – and switches the flows to normal Reno behaviour, roughly doubling the share to 43%. It is still below the fair 50%; this is the main limitation of the design, shared by every delay-based TCP (Vegas obtains about 34% in the same test).

Step 7: The Model Inside the Sender

Procedure: open the *How it works* tab.

THE MODEL · A 4-LEAF DECISION TREE, EVALUATED EVERY 100 MS

RTT ÷ MIN RTT	RTT ÷ ITS 1 S AVERAGE	PREDICTION, 0.5 S AHEAD	ACTION
≤ 1.23	–	Low	Normal Reno growth
> 1.23 (queue building)	≤ 0.85 (queue draining fast)	Medium	Grow at 25% of Reno's rate
> 1.23 (queue building)	**> 0.85**	**High**	**Cut cwnd by 20%, at most once per RTT; stop growing**

Figure 10: The complete learned model: a decision tree with four leaves, evaluated every 100 ms.

The same tab shows the C++ code generated automatically from the trained tree (`ns3/predictive-model.h`, unused parameters omitted):

```
// Generated by analysis/export_tree.py -- do not edit by hand.
// Decision Tree: depth 2, 4 leaves. Returns 0 = Low, 1 = Medium, 2 = High.
inline int
PredictCongestion(double rtt_ratio, /* ...unused features... */ double rtt_vs_avg)
{
    if (static_cast<float>(rtt_ratio) <= 1.2333337664604187f)
    {
        return 0; // Low
    }
    else
    {
        if (static_cast<float>(rtt_vs_avg) <= 0.8548317551612854f)
        {
            return 1; // Medium
        }
        else
        {
            return 2; // High
        }
    }
}
```

If the RTT is more than 23% above the base RTT and is not falling quickly, the tree predicts that the queue will be full within 0.5 s, and the sender cuts its window by 20%. The tree was trained on 106,354 samples, and the generated C++ gives exactly the same answers as the Python model on all 42,235 test samples. It needs no machine-learning library at run time, so it adds almost no cost to the sender.

SECTION 4: Summary of the Demonstration

Table 6: Results shown in the demonstration (seed 4)

Step	Scenario	Reno	Predictive TCP	Result
1	Terminal, high load	1,608 drops	165 drops	≈90% fewer drops
2	100 ms log at t = 30 s	queue 44, RTT 90–103 ms	queue 13, RTT 61–67 ms	short queue, low delay
3	App, high load	100.6 ms, 1,595 drops	63.2 ms, 165 drops	−37.2% RTT, −90% drops
4	All five baselines	–	lowest RTT and queue	BBR 7,871 drops; Vegas 102 ms
5	100-packet queue	144.4 ms	70.0 ms	−51.5% RTT (bufferbloat)
6	4 vs 4 with CUBIC	–	42.9% (21.5% without detector)	below the fair 50%
7	Model	–	4-leaf tree in C++	explainable, near-zero cost

These single-run results match the averages over all ten seeds reported in the project: compared with Reno, Predictive TCP has **93% and 90% fewer drops** and **28% and 36% lower RTT** at medium and high load, with **97–100% of Reno's goodput**. It had fewer drops and a lower RTT than Tahoe, Reno, CUBIC and BBR in all ten seeds, and every drop and RTT improvement over Reno and CUBIC is statistically significant (paired t-test, $p < 0.0001$).

Assumptions for this demonstration:

- All results come from the NS-3.48 simulator on one dumbbell topology with a FIFO queue; real networks may behave differently.
- Every run uses seed 4, which was never used for training, model selection or tuning.
- The terminal summary line totals all received bytes and drops directly, while the app computes them from the 100 ms log, so the two differ by 1–2% for the same run (for example 1,608 vs 1,595 Reno drops).
- Runs marked "from cache" in the app reuse an identical earlier simulation; re-running gives the same result, because every simulation is deterministic for a given seed.

Conclusion: The demonstration shows that predicting congestion half a second ahead lets a TCP sender keep the router queue short: about **90% fewer packet drops** and **37–52% lower delay** than Reno in these runs, at **99.6–99.9% of Reno's goodput**, with a model small enough to read and to run inside the sender. The honest trade-off is fairness against aggressive loss-based senders: with the competition detector Predictive TCP obtains about **43%** of the link against CUBIC, below the fair 50%.